



## Design and Build 2025

# MazeBot

---

**Group**

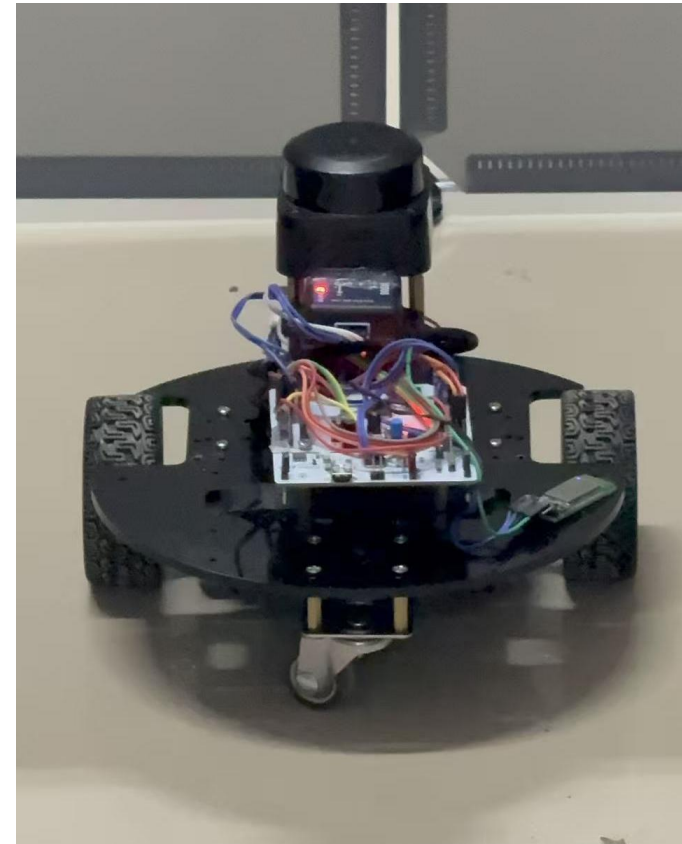
**NO. 17**

**Members**

Yifan Chen  
Zixuan Wang  
Congbo Li  
Mengdi Yang  
Cheng Wei  
Yunlu Zhang  
Yanda Shao  
Lirui Pan  
Siyan Wu  
Fuyang Li  
Kuangyu Chen

# □ Brief Introduction

MazeBot is an intelligent maze exploration robot system based on STM32, integrating **fully independently designed** technologies such as lidar SLAM, automatic route finding & mapping, precise motion control, and real-time visualization.



# □ Our Innovation

The mapping algorithm of the upper computer was all handwritten by myself and did not call any Python libraries

A perfect UI interface and a parameter adjustment system tailored to different scenarios

An accurate mileage calculation method was obtained by combining the robot's own mechanical parameters, motor parameters and kinematic formulas



# Content

1

Autonomous Robot  
Hardware Architecture

2

Firmware Technical Details

3

Upper-Lower Computer  
Collaboration Architecture

4

Software Technical  
Details

5

System integration and  
optimization

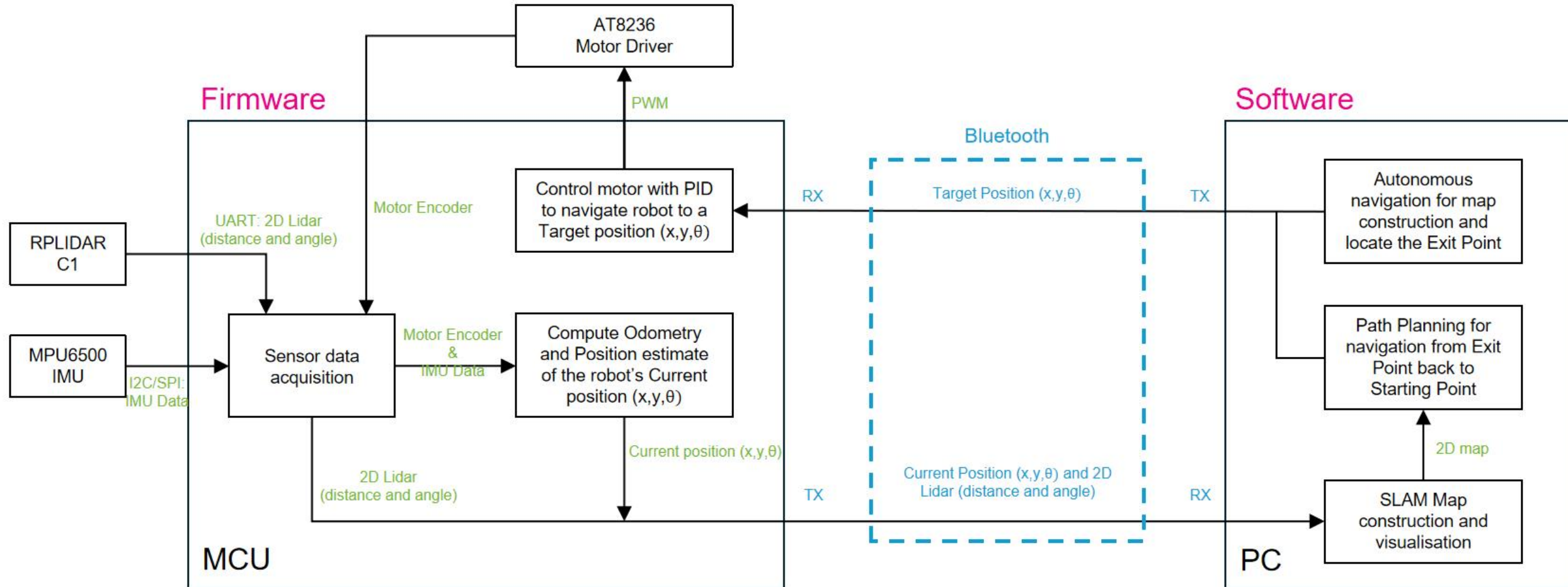
6

Epilogue

# **Part 01**

## **Autonomous Robot Hardware Architecture**

# □ Design Process



# ❑ Hardware Architecture

## 1.1 Main Controller

STM32 Microcontroller - Core Processing Unit  
FreeRTOS RTOS - Real-time Multi-tasking

### **Task Priority Management:**

Priority 4 (Highest): Bluetooth Control, Motor Control

Priority 3 (High): IMU Data Acquisition, LiDAR Processing

Priority 1-2 (Medium/Low): Auxiliary Tasks

## 1.3 Communication System

**HC-04 Bluetooth:** 921600 Baud Rate

### **Multi-UART Architecture:**

Bluetooth: Host Communication

IMU: Sensor Data

LiDAR: Scan Data



## 1.2 Sensor System

### **RPLidar C1 Laser Radar**

Scan Rate: 10Hz

Range: 0.1-3.5m

Resolution: 360°/rotation

Protocol: Custom Binary

### **IM948 9-Axis IMU**

Update Rate: 250Hz

Data Output: Euler Angles, Spatial Displacement, Quaternions

Feature: Magnetometer Fusion

### **Encoder System**

Independent Wheel Encoders

390 Pulses/Revolution (13 lines × 30 ratio)

Interrupt-driven Counting

# **Part 02**

## **Firmware Technical Details**



# □ LiDAR Module (LidarModule.h)

## Core Functions

**Data Packet Parsing:** 5-byte RPLidar protocol parsing

**Quality Filtering:** Filter low-quality points (quality < 5)

**Distance Correction:**

Linear scaling factor: 0.79

Non-linear close-range correction: Correct distortion within 0.5m

**Data Fusion:** LiDAR data and odometry data fusion transmission

## Technical Features

// Data packet structure (18 bytes)

```
struct BluetoothLidarPacket {  
    uint16_t header;           // 0xAA55  
    uint32_t angle_deg_q8;     // Angle × 256  
    uint16_t distance_mm;     // Distance (mm)  
    uint8_t quality;           // Quality (0-63)  
    int16_t odom_x_cm;         // Odometry X (cm)  
    int16_t odom_y_cm;         // Odometry Y (cm)  
    int32_t odom_theta_deg_q8; // Orientation × 256  
    uint8_t checksum;          // XOR checksum  
};
```

```
// 蓝牙数据包格式 (雷达 + Odom融合)  
struct __attribute__((packed)) BluetoothLidarPacket {  
    uint16_t header;           // 固定头: 0xAA55  
    uint32_t angle_deg_q8;     // 角度 * 256 (uint32_t避免360°*256=92160溢出uint16_t)  
    uint16_t distance_mm;     // 距离 mm  
    uint8_t quality;           // 质量  
    int16_t odom_x_cm;         // 里程计X (cm)  
    int16_t odom_y_cm;         // 里程计Y (cm)  
    int32_t odom_theta_deg_q8; // 里程计Theta (度*256) - 使用int32_t避免溢出  
    uint8_t checksum;          // 校验和  
};
```



# ❑ Motor Control Module (MotorControl.h)

## PID Control System

### Three-Layer PID Architecture

Heading Loop: Angle Error → Target Speed Difference

Speed Difference Loop: Speed Difference → Duty Cycle Difference

Steering Loop: Angle Error → Duty Cycle

### PID Parameters

```
// 航向环 PID
#define KH_P 2.0
#define KH_I 0.0
#define KH_D 0.1

// 速差环 PID
#define KV_P 3.0
#define KV_I 0.0
#define KV_D 0.1

// 转向环 PID
#define KT_P 0.4    // 转向P: 降低以减少超调
#define KT_I 0.0    // 转向I: 消除稳态误差
#define KT_D 10.0   // 转向D: 增大以抑制震荡
```

## Motion Control Modes

Control Strategies

Open-Loop Control: Basic direction control

Closed-Loop Control: Precise distance/angle control

## Precision Commands

F50: Forward 50cm

L90: Turn left 90°

R45: Turn right 45°



# ❑ Encoder Odometry Module (EncoderModule.h)

## Differential Wheel Odometry Algorithm

### Physical Parameters

cpp

```
#define WHEEL_DIAMETER 0.065 // Wheel diameter 65mm
#define WHEEL_BASE 0.100 // Wheel base 100mm
#define TICKS_PER_REV 390 // Pulses per revolution
#define ENCODER_SCALE_FACTOR 0.534 // Accuracy correction factor
```

```
// 小车物理参数 (需要实测后填入准确值)
#define WHEEL_DIAMETER 0.065 // 轮径 (米) - 默认65mm, 请实测后修改
#define WHEEL_BASE 0.100 // 轮距 (米) - 默认100mm, 请实测后修改
#define TICKS_PER_REV 390 // 编码器每圈脉冲数 (13线x30减速比=390)
#define DEG_TO_RAD 0.017453292519943295 // 度转弧度常量 (PI/180)

// 编码器里程计缩放系数 (用于微调精度)
#define ENCODER_SCALE_FACTOR 0.534 // 缩放系数, 1.0=无缩放, >1.0=放大, <1.0=缩小
```

## Calculation Principle

### 4-Step Processing Pipeline

Read Encoder Increments - Capture pulse counts from both wheels

Convert to Linear Distance - Transform pulses to actual wheel movement

Differential Calculation - Compute robot center movement using differential drive kinematics

World Coordinate Update - Combine with IMU orientation for global position tracking



# ❑ IMU Data Processing (im948\_CMD.h)



## Data Fusion Strategy

### Multi-Sensor Integration

Encoder Odometry: High-precision short-distance displacement

IMU Orientation: Accurate angle information

Timestamp Synchronization: 100-data point circular buffer



## Independent Odom Transmission

### High-Performance Data Stream

11-byte Independent Packet (0xBB55 header)

250Hz High-Frequency Update

Zero-Delay Pose Transmission



# **Part 03**

## **Upper-Lower Computer Collaboration Architecture**

# ❑ Role Split: STM32 vs Python

A clean division of labor for a tightly coupled system.



## Lower Computer (STM32)

**Real-time Sensor Acquisition:** Collects raw data from IMU, LiDAR, and encoders.

**Motor PID Control:** Executes precise, real-time motion commands.

**Encoder Odometry:** Calculates robot's position and orientation.

**Bluetooth Communication:** Handles low-level data transmission and command responses.



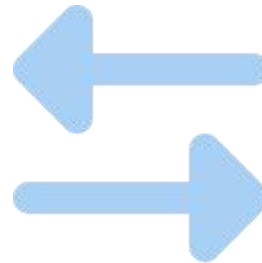
## Upper Computer (Python)

**Live Visualization:** Renders sensor data and map in real-time.

**SLAM Algorithm:** Constructs and updates the environment map.

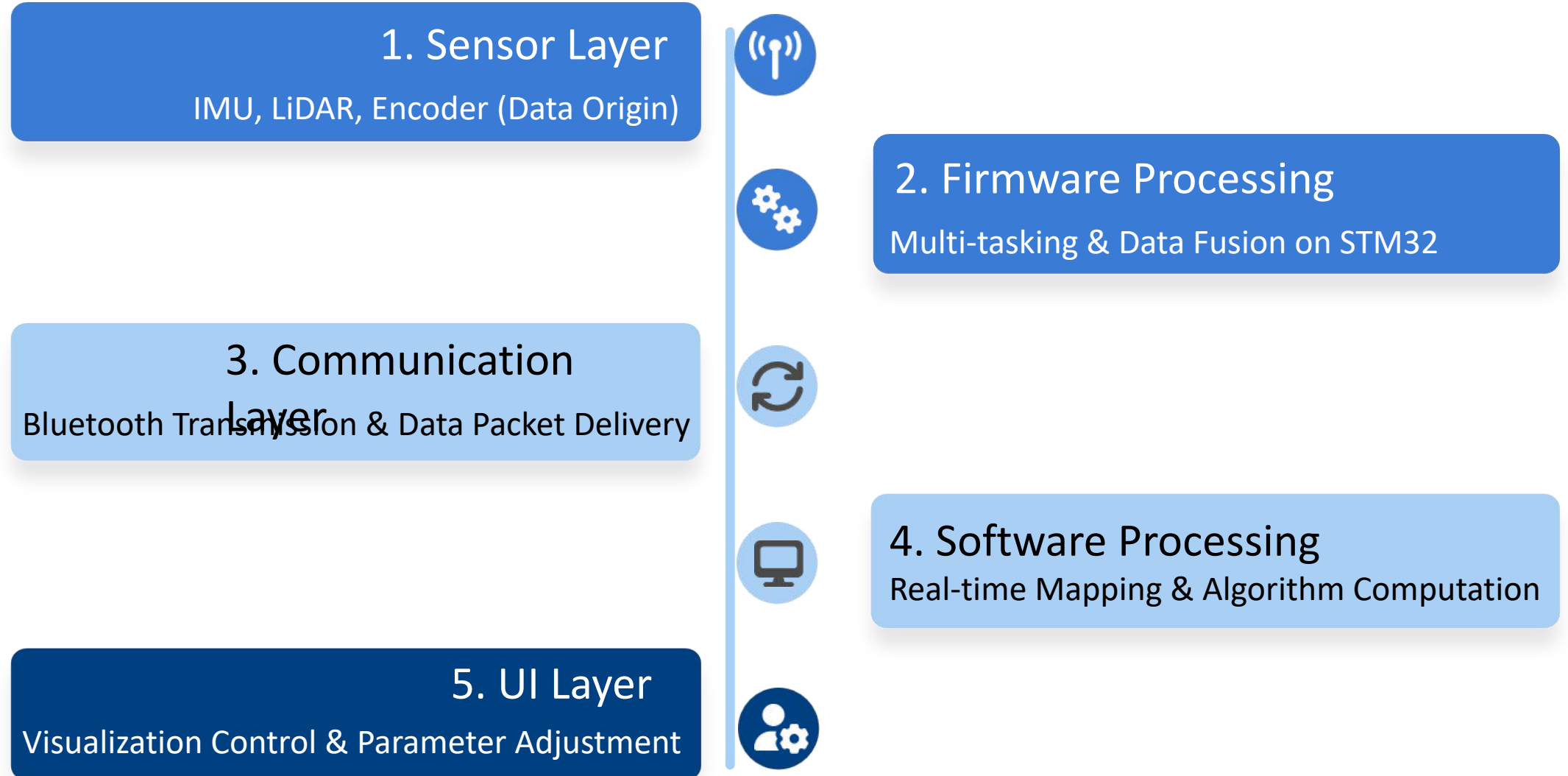
**Path Planning:** Makes high-level navigation decisions.

**User Interface:** Provides controls for parameter tuning and robot command.



# □ Layered Data Flow Pipeline

From raw sensors to actionable UI, a journey through the system's core.



# □ Lightweight Packet Blueprint

---

## LiDAR Fusion Packet

For scan-synced odometry.

Header: 0xAA55

Payload: Angle, Distance, Quality,  
Odometry

Size: 18 bytes

Frequency: 10 Hz

---

## Standalone Odom Packet

For high-rate localization.

Header: 0xBB55

Payload: Position, Orientation

Size: 11 bytes

Frequency: 250 Hz

---

## Control Command Packet

For motion and system tasks.

Format: ASCII String

Examples: W/S/A/D, F50, L90, x, S

Size: Variable

Type: Human-readable





# ❑ Realtime & Sync Tactics

## Guaranteeing Real-Time Performance

### STM32 Firmware

Utilizes a **priority-based RTOS** to ensure critical tasks (motor control, odometry) are executed with minimal jitter.

### Python Software

Employs a **dedicated RX thread** to prevent data loss and maintain UI responsiveness during heavy computation.

### Communication

UART baud rate is maximized to **921600** to create a high-throughput, low-latency link.

## Maintaining Data Synchronization

### Timestamp Alignment

**MU data is buffered and synchronized with LiDAR scan data** to ensure a consistent snapshot of the robot's state.

### Pose Interpolation

**High-frequency odometry** allows for pose correction even as the LiDAR scan is being received, preventing map distortion.

### Coordinate Unification

**Automatic conversion** from the LiDAR's left-hand coordinate system to the screen's right-hand system is performed.



# **Part 04**

## **Software Technical Details**

# ❑ Visualizer Engine Architecture

A **multi-threaded design** in lidar\_visualizer.py for maximum performance.



## 1. Data Reception Thread

An **independent thread** solely responsible for reading and parsing data from the serial port to prevent I/O blocking.



## 2. Main Thread

Handles **real-time visualization rendering and SLAM processing**, ensuring the GUI remains fluid and responsive.



## 3. Shared Odom Data

A **thread-safe buffer** that allows the RX thread to deposit odometry data for the main thread to consume without race conditions.

## Performance Metrics

166Hz

UI Update Frequency

15k

Data Packets per Frame

Independent thread  
processing

Multi-Core Utilization



# ❑ SLAM Algorithm (SimpleGridSLAM Class)

## Coordinate system unification

```
# 坐标系转换
# RPLidar C1: 顺时针 → 逆时针镜像
angle_deg = 360.0 - angle_deg
# 显示变换: 逆时针90°
x_display = -y_orig
y_display = x_orig
```

## Intelligent filtering

Rotation detection filtering: Prevents trailing in mapping during rotation

Motion prediction filtering: Eliminate historical point trailing

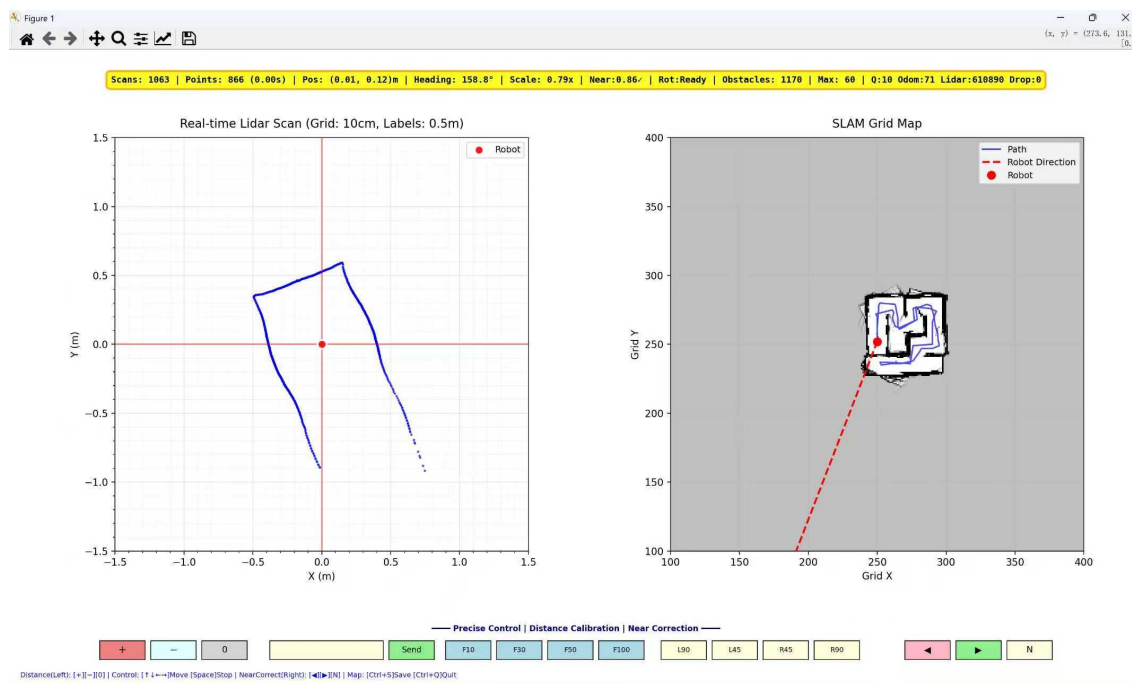
Overlap protection mechanism: Protecting the characteristics of the real environment

## Grid map construction

Map size: 500×500 raster

Resolution: 5cm per grid

Probability Update: Cumulative weight of obstacles



# □ Frontier Detection & Selection Logic



## Data Preprocessing

LiDAR data undergoes dual distance correction, quality filtering, and time-windowing.



## Environmental Perception

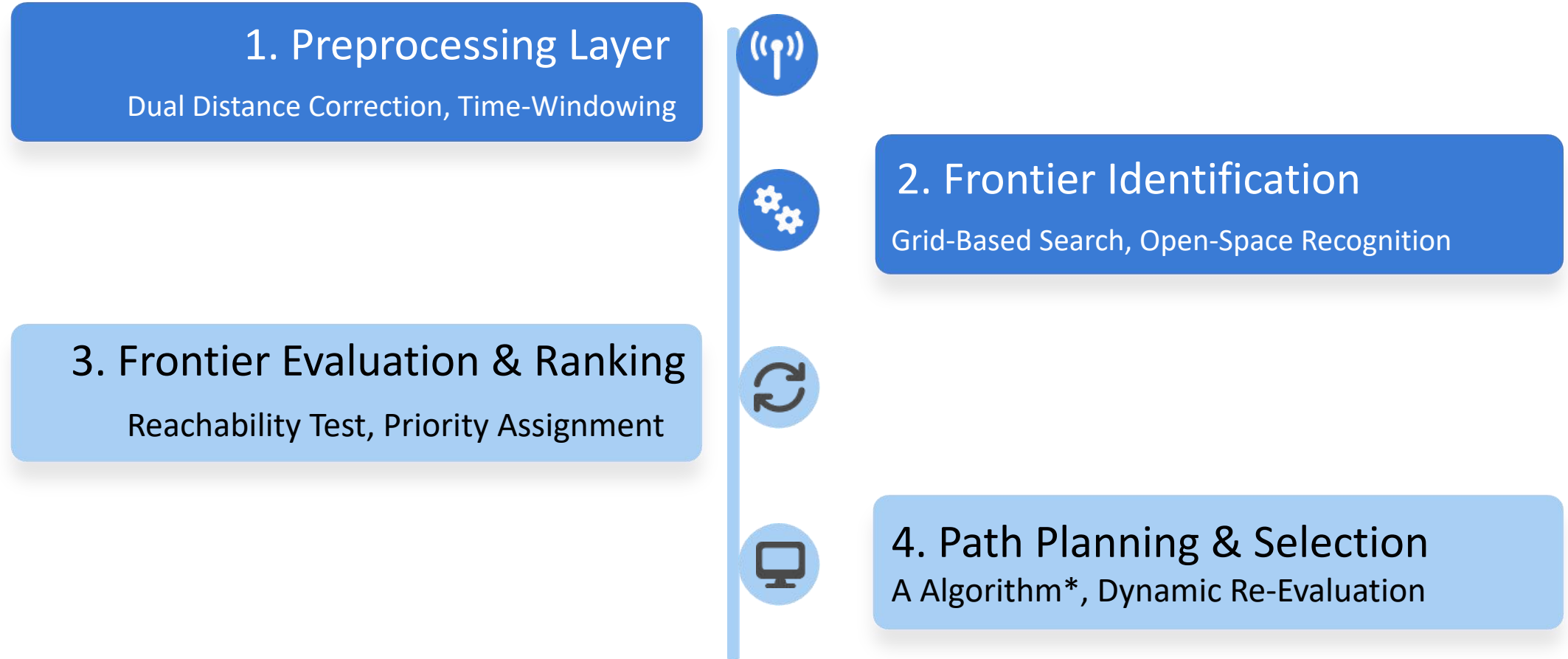
Detects open areas (>5m) as potential exits and recognizes obstacles from point clouds.



## Path Planning

The A\* algorithm plans the optimal route to the selected frontier on the grid map.

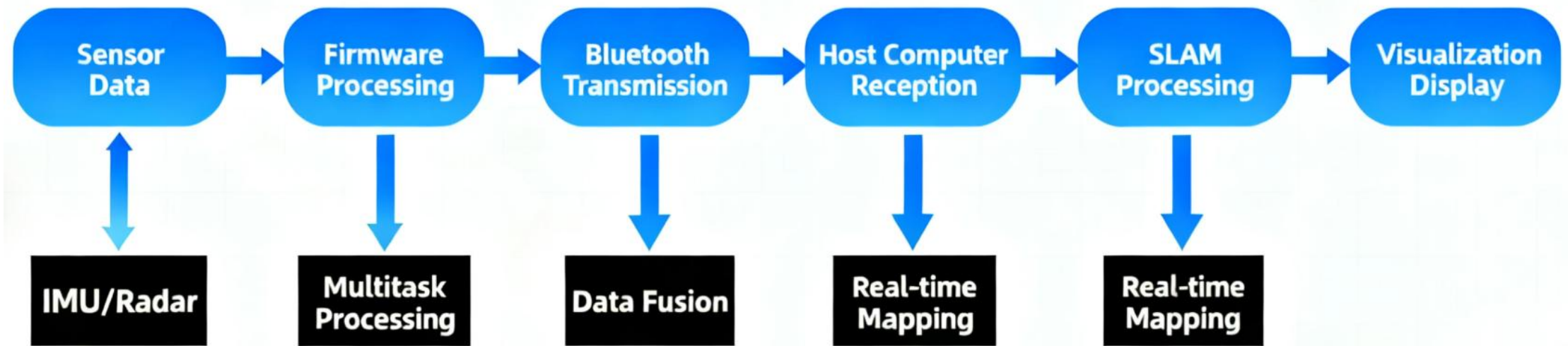
# ❑ Frontier Detection & Selection: Detailed Explanation



# **Part 05**

## **System integration and optimization**

# ❑ System integration and optimization



## Performance Indicators

Real-time performance: **<5ms** position update delay

Accuracy: **±1cm/ m** displacement accuracy

Stability: **250Hz** IMU update, **10Hz** radar scan

Robustness: Multi-level data checksum and error recovery





# **Part 06**

# **Epilogue**

# Design and Build Group 17

Zixuan Wang, Yifan Chen, Cheng Wei, Mengdi Yang, Congbo Li, Yunlu Zhang  
Yanda Shao, Lirui Pan, Siyan Wu, Fuyang Li, Kuangyu Chen

# □ Task Allocation

Lower

| Member      | Class      | BUPT ID    | Task Allocation                                                                                                                                                                                                          |
|-------------|------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Yifan Chen  | 2023215103 | 2023213098 | Coordinate progress, design wiring schemes, connect and check hardware issues, design FreeRTOS tasks and debugging programs for each sensor, consult hardware manuals and function descriptions, and write documents.    |
| Zixuan Wang | 2023215101 | 2023213051 | Assist in progress management, write documents, collect and integrate code, hardware manuals and other materials, and assist in function testing and vehicle assembly.                                                   |
| Cheng Wei   | 2023215105 | 2023213171 | Check program logic, inspect hardware circuits, design and debug motion control logic, design IMU and radar data collection, calculation and filtering functions, and record serial port debugging data.                 |
| Mengdi Yang | 2023215107 | 2023213226 | Collect and integrate code, hardware manuals and other materials, collect and integrate key hardware driver library functions and guide team members to use them, assist in writing documents, and beautify the vehicle. |
| Congbo Li   | 2023215109 | 2023213269 | Inspect, test and repair hardware circuits, assemble and repair vehicles, assist in manual and information consultation, assist in testing motion functions and parameter adjustment, and assist in writing documents.   |
| Yunao Zhang | 2023215117 | 2023213519 | Assemble and repair vehicles, check program logic, inspect hardware circuits, assist in testing motion functions and parameter adjustment, and assist in writing documents.                                              |

Upper

| Member       | Class      | BUPT ID    | Task Allocation                                                                                                                                                                                                                                                                                                                                                                                |
|--------------|------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fuyang Li    | 2023215122 | 2023213632 | Responsible for software overall architecture design and development planning; coordinate the integration of automatic navigation main program, path planning, obstacle avoidance and mapping modules; design task scheduling and system communication framework; be responsible for overall debugging and performance verification; write main software description and acceptance documents. |
| Lirui Pan    | 2023215113 | 2023213401 | Responsible for the implementation of automatic navigation main logic and integration of core algorithms; lead the development and parameter adjustment of the A* path planning module; implement path planning, obstacle detection and target state machine logic; assist in constructing the return path reuse mechanism.                                                                    |
| Yanda Shao   | 2023215111 | 2023213335 | Responsible for the development of SLAM mapping and localization module; fuse laser radar and IMU data to realize map construction and position estimation; design map update, visualization and data interface; verify mapping accuracy and system robustness.                                                                                                                                |
| Siyan Wu     | 2023215115 | 2023213467 | Responsible for the implementation of DWA (Dynamic Window Approach) dynamic obstacle avoidance algorithm; construct speed window and obstacle avoidance cost function based on the car dynamics model; perform obstacle avoidance performance tuning and testing in complex environments.                                                                                                      |
| Kuangyu Chen | 2023215124 | 2023213698 | Responsible for FreeRTOS task scheduling and multi-thread communication; design the synchronization mechanism between sensor data collection, control commands and planning tasks; debug task priority and system real-time performance; assist in the integration and joint debugging test of the whole system.                                                                               |





**Thank You**